Pre_processing_FeatureEngineering_Modeling_Pipeline

December 1, 2025

[10]:
```
!pip install emoji
import pandas as pd

# read sample data from csv
file_path = '/root/data/data/yelp_review_data_binned.csv'
df_review_data_binned = pd.read_csv(file_path, engine="python",␣
 ↪on_bad_lines="skip")

# Null value checking
print(df_review_data_binned.isnull().sum())
df_review_data_binned.info()
```

```
Requirement already satisfied: emoji in ./dsenv/lib/python3.12/site-packages
(2.15.0)
review_id         0
user_id           0
business_id       0
stars             0
useful            0
funny             0
cool              0
text              0
date              0
stars_business    0
stars_binned      0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4641 entries, 0 to 4640
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   review_id       4641 non-null   object
 1   user_id         4641 non-null   object
 2   business_id     4641 non-null   object
 3   stars           4641 non-null   int64
 4   useful          4641 non-null   int64
 5   funny           4641 non-null   int64
 6   cool            4641 non-null   int64
```

```
 7   text            4641 non-null   object
 8   date            4641 non-null   object
 9   stars_business  4641 non-null   float64
 10  stars_binned    4641 non-null   int64
dtypes: float64(1), int64(5), object(5)
memory usage: 399.0+ KB
```

[11]:
```
# Pre-processing
import emoji
import re
import nltk
import unicodedata
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

df_review_data_binned = df_review_data_binned.dropna(subset=['stars_binned'])
df_review_data_binned = df_review_data_binned.dropna(subset=['text'])
print(df_review_data_binned.shape)


# Identify if there are any emoji characters

def clean_text(text):
  if text is None:
    return ""
  text = emoji.replace_emoji(text, replace='')
  # convert to lower case
  text = text.lower()
  # remove strings that start with http
  text = re.sub(r'http\S+', '', text)
  # remove HTML tags
  text = re.sub(r'<.*?>', '', text)
  # remove special characters
  text = re.sub(r'www\S+', '', text)
  # remove punctuations
  text = re.sub(r'[^\w\s]', '', text)
  # remove numbers
  text = re.sub(r'\d+', '', text)
  text = unicodedata.normalize('NFKD', text)
  text = text.encode('ascii', 'ignore').decode('utf-8')
  #remove any left non-ascii characters
  text = re.sub(r'[^a-zA-Z\s]', '', text)
  # remove extra spaces
  text = re.sub(r'\s+', ' ', text).strip()

  return text
```

```
df_review_data_binned["text"] = df_review_data_binned["text"].apply(clean_text)
# download stopwords
nltk.download('stopwords')
# download tokenizer data
nltk.download('punkt_tab')
# remove stopwords
stop_words = set(stopwords.words('english'))
# remove stopwords from text column
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣
 ↪'.join([word for word in x.split() if word not in (stop_words)]))
# perform lemitazation
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
# download lexical database
nltk.download('wordnet')
# multilingual lexical database
nltk.download('omw-1.4')
# perform lemmatization
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣
 ↪'.join([lemmatizer.lemmatize(word) for word in word_tokenize(x)]))
```

```
(4641, 11)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data…
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data…
[nltk_data]   Package omw-1.4 is already up-to-date!
```

[12]:
```
from sklearn.model_selection import train_test_split
# define X and y
X = df_review_data_binned[["text","useful"]]
y = df_review_data_binned['stars_binned']

# Split all data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Take only text
X_train_text = X_train['text']
X_test_text = X_test['text']
```

[17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
```

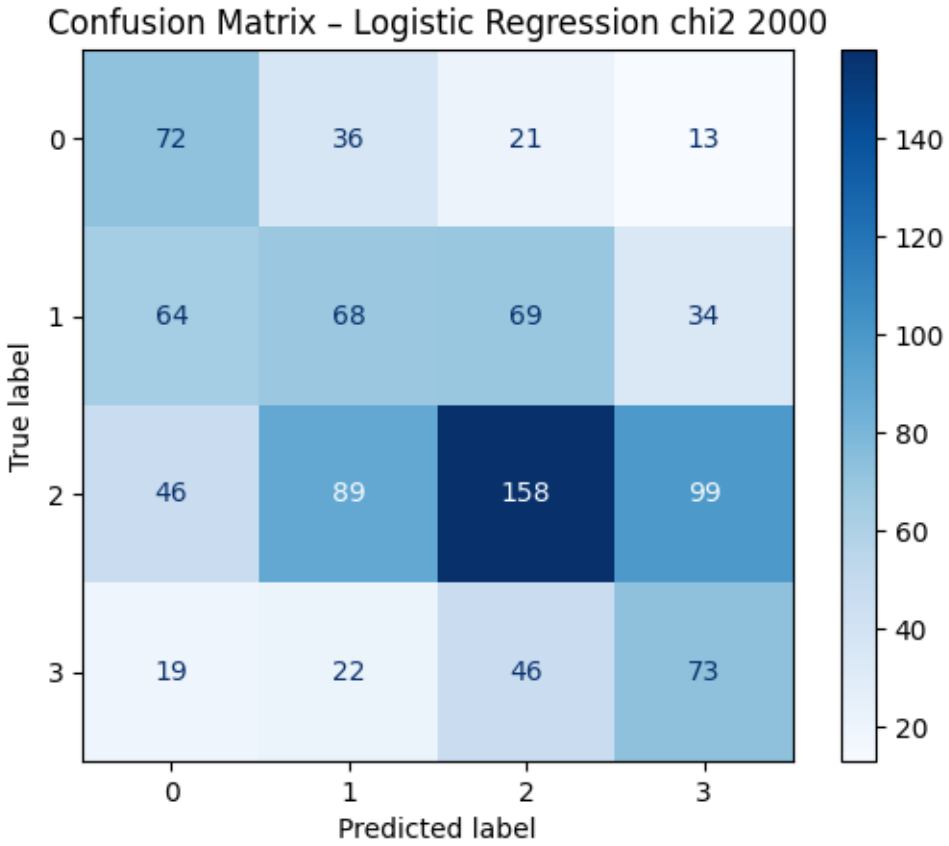
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
)
import random
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

k_values = [2000, 5000]
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        n_jobs=-1,
        solver="saga",
        penalty="l2",
        class_weight="balanced",
    ),
    "Naive Bayes": MultinomialNB(),
    "Decision Tree": DecisionTreeClassifier(
        criterion="gini",
        max_depth=20,
        min_samples_leaf=5,
        random_state=42,
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced",
        max_depth=None,
        n_jobs=-1,
        random_state=42,
    ),
    "Linear SVC": LinearSVC(class_weight="balanced"),
}
def plot_confusion_matrices(model_name, y_test, y_pred, k_value):
  """
  Plots confusion matrices for Models
  """
```

```
  cmaps = [
    "Blues", "Greens", "Purples", "Oranges", "Reds",
    "viridis", "plasma", "inferno", "magma", "cividis",
    "YlGnBu", "YlOrRd", "PuBuGn", "GnBu", "BuPu",
  ]

  cmap_choice = random.choice(cmaps)

  cm = confusion_matrix(y_test, y_pred)
  disp = ConfusionMatrixDisplay(confusion_matrix=cm)
  disp.plot(cmap=cmap_choice)
  plt.title(f"Confusion Matrix - {model_name} chi2 {k_value}")
  plt.show()


def evaluate_and_report_model(
    model_name,
    model,
    k_value,
    X_train_text,
    y_train,
    X_test_text,
    y_test,
    skfold,
):
    tfidf_vectorizer = TfidfVectorizer(
        min_df=3,
        max_df=0.9,
        max_features=10000,
        ngram_range=(1, 2),
    )

    feature_selector = SelectKBest(score_func=chi2, k=k_value)

    pipeline = Pipeline(
        [
            ("tfidf", tfidf_vectorizer),
            ("chi2", feature_selector),
            ("clf", model),
        ]
    )

    cv_scores = cross_val_score(
        pipeline,
        X_train_text,
        y_train,
        cv=skfold,
```

```
        scoring="f1_macro",
        n_jobs=-1,
    )
    print(f"CV {model_name} (k={k_value}) F1 macro per fold: {cv_scores}")
    print(
        f"CV {model_name} (k={k_value}) "
        f"mean={cv_scores.mean():.4f}, std={cv_scores.std():.4f}"
    )

    pipeline.fit(X_train_text, y_train)
    y_pred = pipeline.predict(X_test_text)

    print(f"\nEvaluation Metrics {model_name} chi2 k = {k_value}")
    print(f"{model_name} Accuracy: {accuracy_score(y_test, y_pred):.4f}")
    print(
        "Classification report:\n",
        classification_report(y_test, y_pred, zero_division=0),
    )
    print(
        "Macro F1-score:",
        f1_score(y_test, y_pred, average="macro", zero_division=0),
    )
    print(
        "Weighted F1-score:",
        f1_score(y_test, y_pred, average="weighted", zero_division=0),
    )

    plot_confusion_matrices(model_name, y_test, y_pred, k_value)

for k_value in k_values:
    for model_name, model in models.items():
        evaluate_and_report_model(
            model_name,
            model,
            k_value,
            X_train_text,
            y_train,
            X_test_text,
            y_test,
            skfold,
        )
```

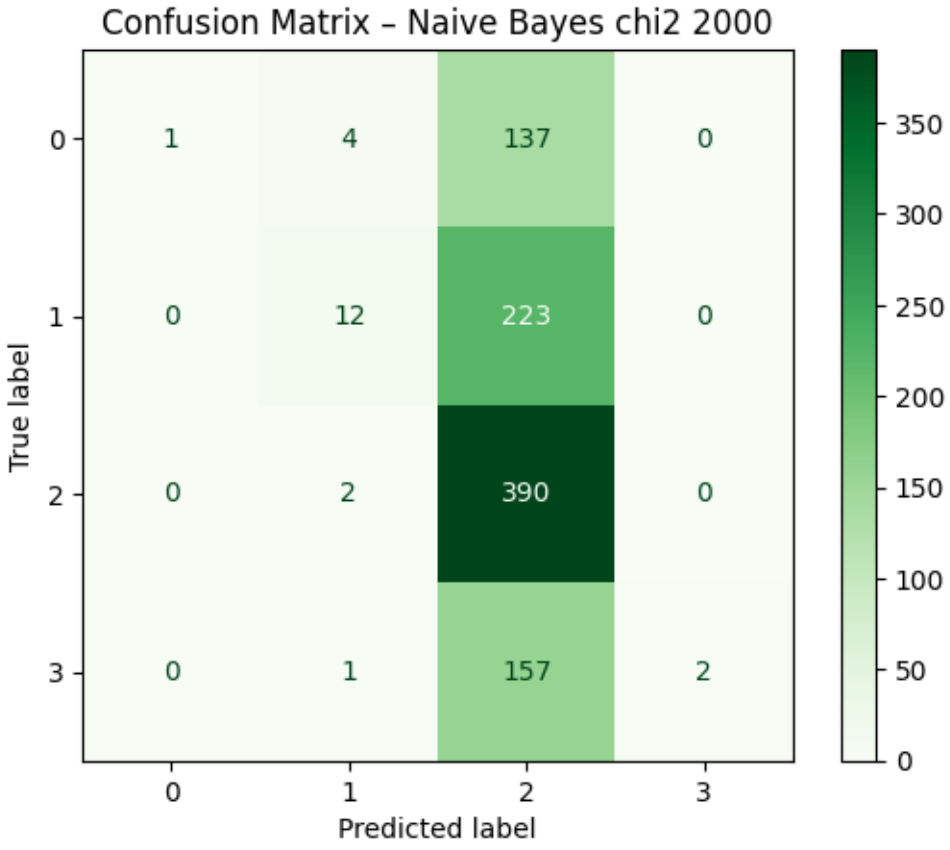
```
CV Logistic Regression (k=2000) F1 macro per fold: [0.38006953 0.40246924
0.4045189  0.38832375 0.40744072]
CV Logistic Regression (k=2000) mean=0.3966, std=0.0105

Evaluation Metrics Logistic Regression chi2 k = 2000
```

```
Logistic Regression Accuracy: 0.3994
Classification report:
              precision    recall  f1-score   support

           1       0.36      0.51      0.42       142
           2       0.32      0.29      0.30       235
           3       0.54      0.40      0.46       392
           4       0.33      0.46      0.39       160

    accuracy                           0.40       929
   macro avg       0.39      0.41      0.39       929
weighted avg       0.42      0.40      0.40       929

Macro F1-score: 0.3919782422329404
Weighted F1-score: 0.40133982244189265
```



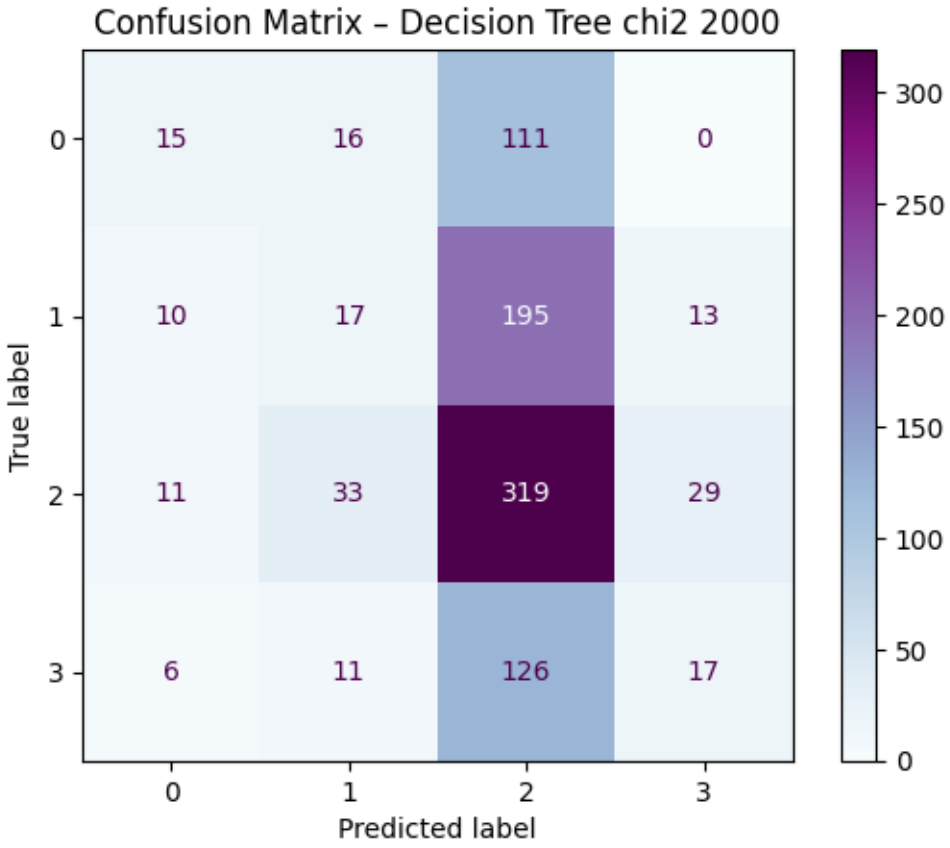

```
CV Naive Bayes (k=2000) F1 macro per fold: [0.15653628 0.16423433 0.15617264
0.15399986 0.15621189]
CV Naive Bayes (k=2000) mean=0.1574, std=0.0035
```

```
Evaluation Metrics Naive Bayes chi2 k = 2000
Naive Bayes Accuracy: 0.4360
Classification report:
              precision    recall  f1-score   support

           1       1.00      0.01      0.01       142
           2       0.63      0.05      0.09       235
           3       0.43      0.99      0.60       392
           4       1.00      0.01      0.02       160

    accuracy                           0.44       929
   macro avg       0.77      0.27      0.18       929
weighted avg       0.67      0.44      0.28       929

Macro F1-score: 0.18340686368787937
Weighted F1-score: 0.2836624521367832
```



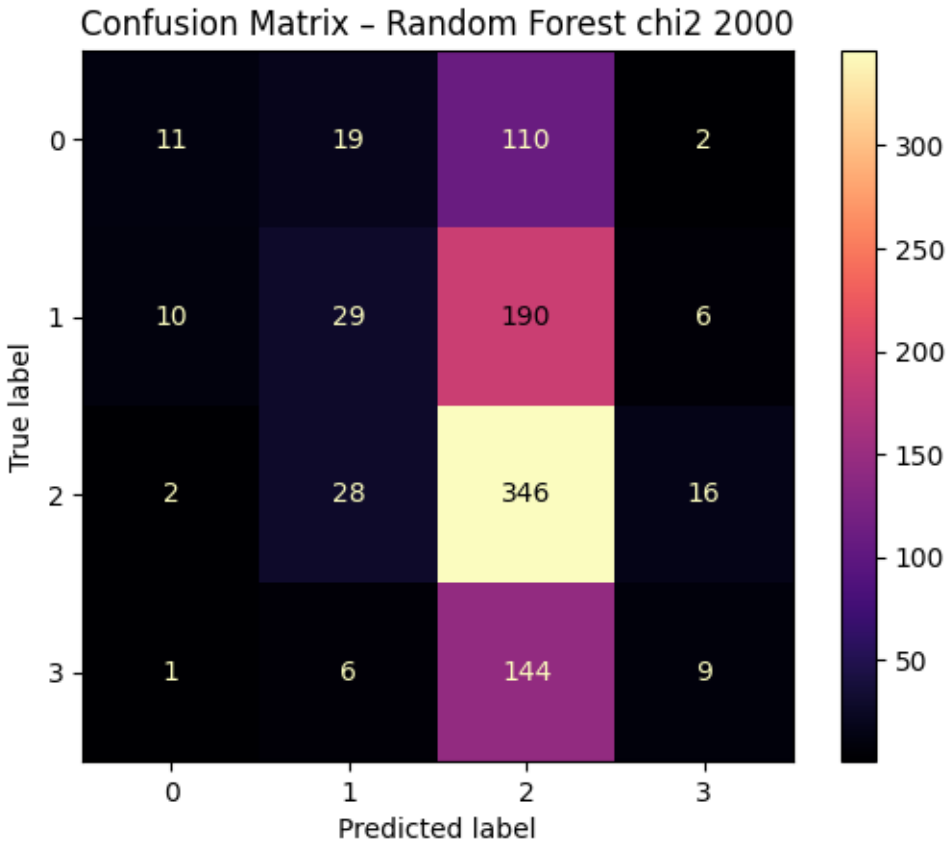

```
CV Decision Tree (k=2000) F1 macro per fold: [0.23976006 0.23888053 0.24669879
0.29960598 0.25848813]
CV Decision Tree (k=2000) mean=0.2567, std=0.0226
```

```
Evaluation Metrics Decision Tree chi2 k = 2000
Decision Tree Accuracy: 0.3961
Classification report:
              precision    recall  f1-score   support

           1       0.36      0.11      0.16       142
           2       0.22      0.07      0.11       235
           3       0.42      0.81      0.56       392
           4       0.29      0.11      0.16       160

    accuracy                           0.40       929
   macro avg       0.32      0.27      0.25       929
weighted avg       0.34      0.40      0.31       929

Macro F1-score: 0.2463623015648265
Weighted F1-score: 0.31475562980546123
```



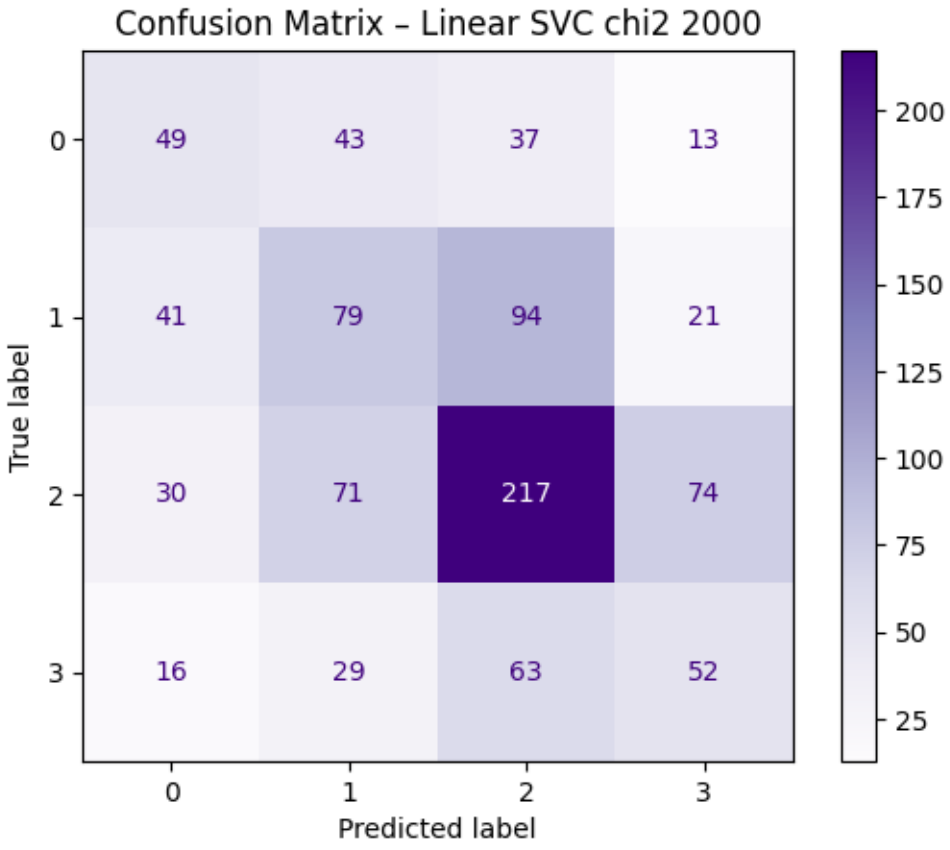

```
CV Random Forest (k=2000) F1 macro per fold: [0.24779957 0.24142174 0.24909133
0.25710358 0.261594  ]
```

```
CV Random Forest (k=2000) mean=0.2514, std=0.0071

Evaluation Metrics Random Forest chi2 k = 2000
Random Forest Accuracy: 0.4252
Classification report:
              precision    recall  f1-score   support

           1       0.46      0.08      0.13       142
           2       0.35      0.12      0.18       235
           3       0.44      0.88      0.59       392
           4       0.27      0.06      0.09       160

    accuracy                           0.43       929
   macro avg       0.38      0.28      0.25       929
weighted avg       0.39      0.43      0.33       929

Macro F1-score: 0.2485520153565313
Weighted F1-score: 0.3296385061448913
```

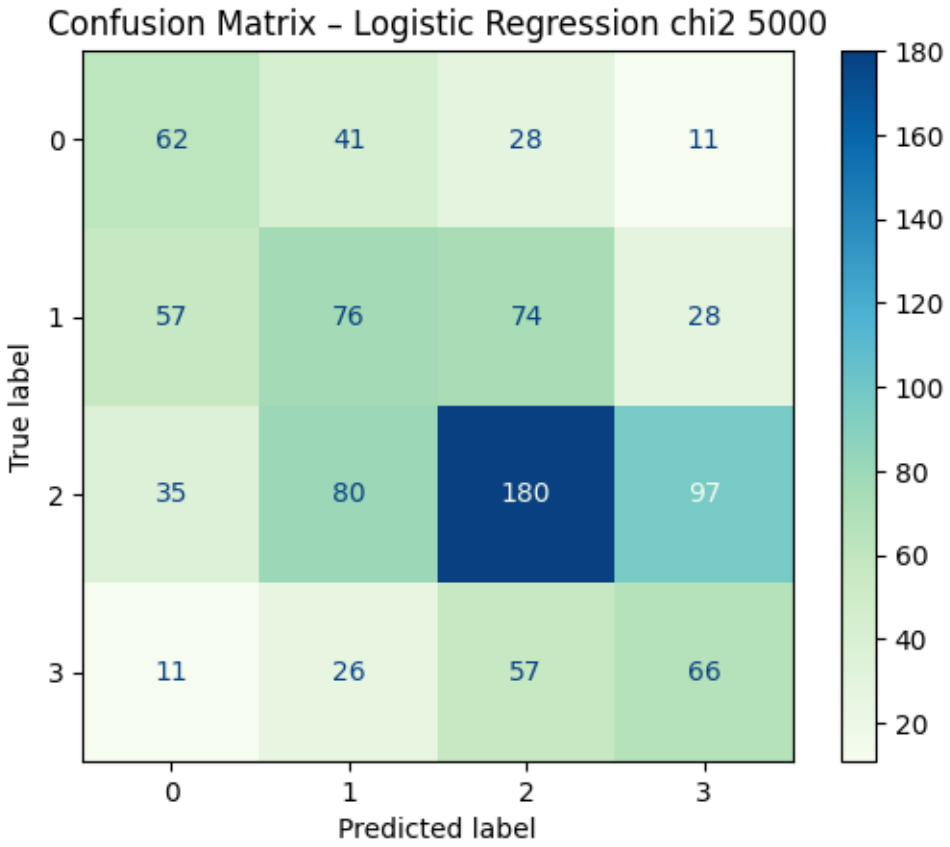



```
CV Linear SVC (k=2000) F1 macro per fold: [0.38998257 0.40945437 0.41200261
```

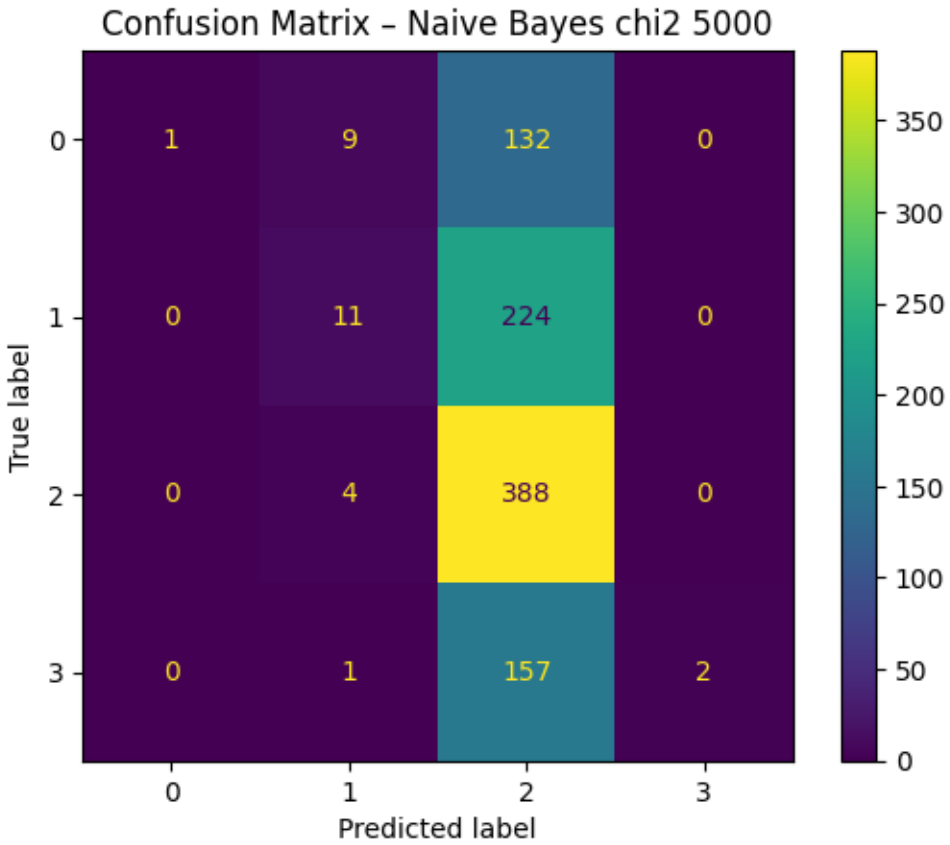
```
0.39397383 0.38387808]
CV Linear SVC (k=2000) mean=0.3979, std=0.0110

Evaluation Metrics Linear SVC chi2 k = 2000
Linear SVC Accuracy: 0.4273
Classification report:
              precision    recall  f1-score   support

           1       0.36      0.35      0.35       142
           2       0.36      0.34      0.35       235
           3       0.53      0.55      0.54       392
           4       0.33      0.33      0.33       160

    accuracy                           0.43       929
   macro avg       0.39      0.39      0.39       929
weighted avg       0.42      0.43      0.43       929

Macro F1-score: 0.39093106314793385
Weighted F1-score: 0.4253717148393221
```



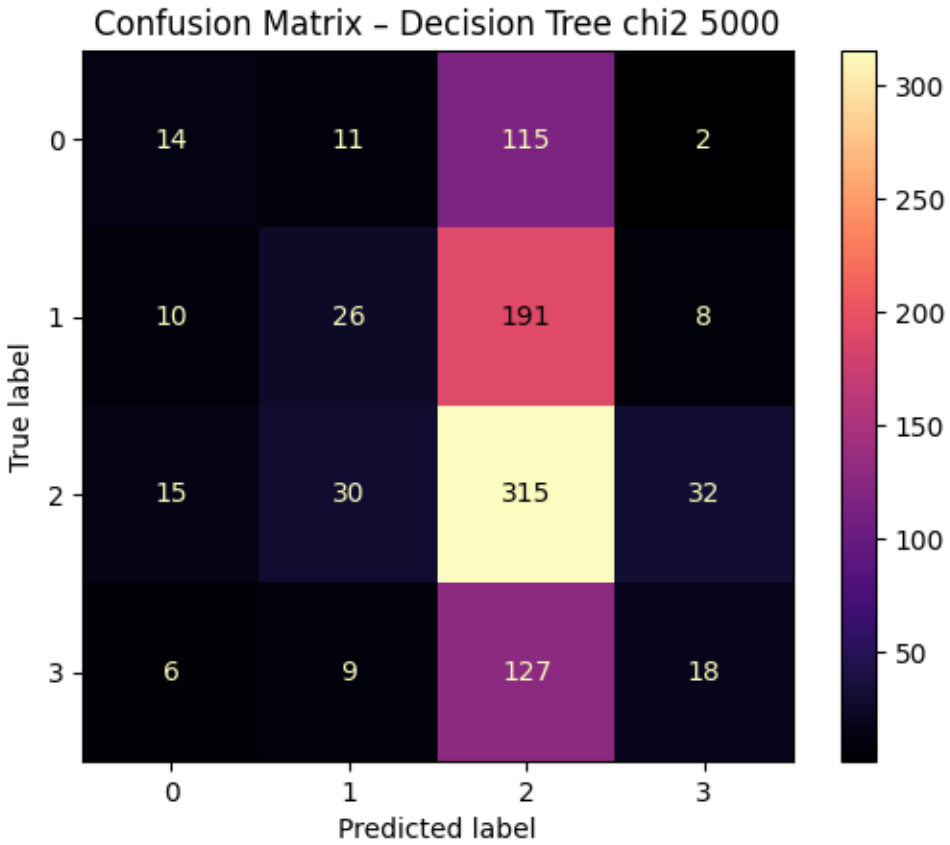
```
CV Logistic Regression (k=5000) F1 macro per fold: [0.40151536 0.4072349
0.39002192 0.40895023 0.38365957]
CV Logistic Regression (k=5000) mean=0.3983, std=0.0099

Evaluation Metrics Logistic Regression chi2 k = 5000
Logistic Regression Accuracy: 0.4133
Classification report:
              precision    recall  f1-score   support

           1       0.38      0.44      0.40       142
           2       0.34      0.32      0.33       235
           3       0.53      0.46      0.49       392
           4       0.33      0.41      0.36       160

    accuracy                           0.41       929
   macro avg       0.39      0.41      0.40       929
weighted avg       0.42      0.41      0.42       929

Macro F1-score: 0.3982258670538336
Weighted F1-score: 0.41629652558344454
```



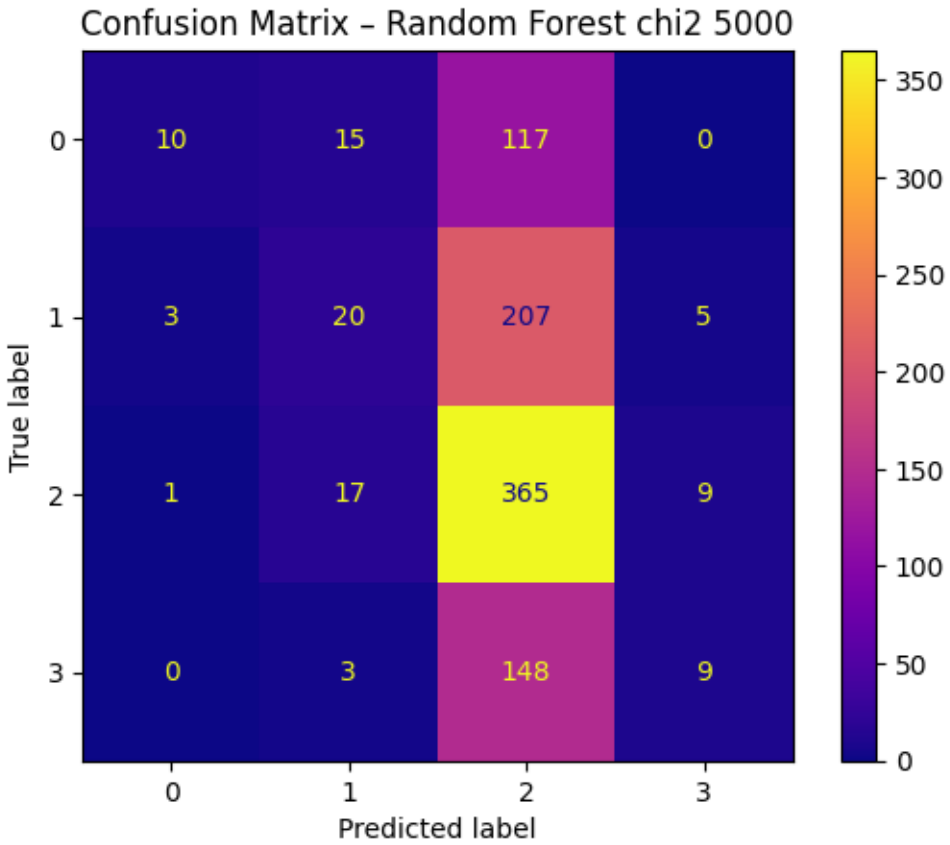
```
CV Naive Bayes (k=5000) F1 macro per fold: [0.16130383 0.16174462 0.16797974
0.15833584 0.15727065]
CV Naive Bayes (k=5000) mean=0.1613, std=0.0037

Evaluation Metrics Naive Bayes chi2 k = 5000
Naive Bayes Accuracy: 0.4327
Classification report:
              precision    recall  f1-score   support

           1       1.00      0.01      0.01       142
           2       0.44      0.05      0.08       235
           3       0.43      0.99      0.60       392
           4       1.00      0.01      0.02       160

    accuracy                           0.43       929
   macro avg       0.72      0.26      0.18       929
weighted avg       0.62      0.43      0.28       929

Macro F1-score: 0.18086185891676998
Weighted F1-score: 0.2810353937983295
```



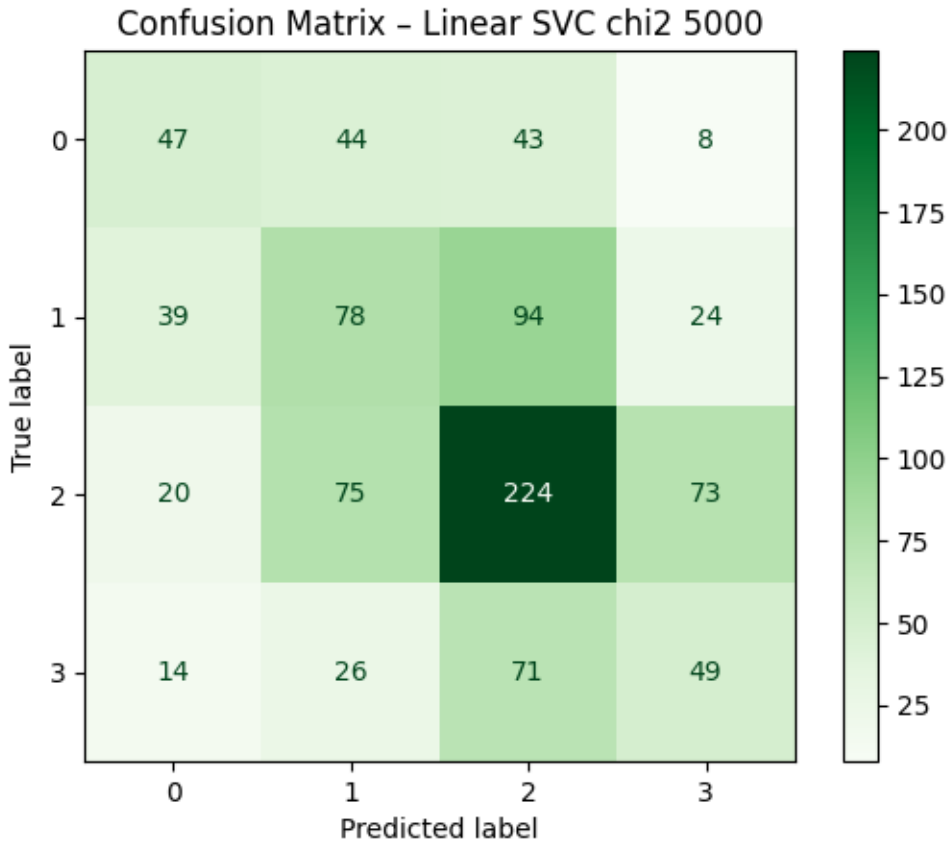
```
CV Decision Tree (k=5000) F1 macro per fold: [0.23944155 0.25708158 0.25354287
0.28215488 0.24540299]
CV Decision Tree (k=5000) mean=0.2555, std=0.0147

Evaluation Metrics Decision Tree chi2 k = 5000
Decision Tree Accuracy: 0.4015
Classification report:
              precision    recall  f1-score   support

           1       0.31      0.10      0.15       142
           2       0.34      0.11      0.17       235
           3       0.42      0.80      0.55       392
           4       0.30      0.11      0.16       160

    accuracy                           0.40       929
   macro avg       0.34      0.28      0.26       929
weighted avg       0.36      0.40      0.33       929

Macro F1-score: 0.2583007838129636
Weighted F1-score: 0.32655331939327864
```



```
CV Random Forest (k=5000) F1 macro per fold: [0.23700805 0.23143448 0.23356468
0.24942128 0.24856058]
CV Random Forest (k=5000) mean=0.2400, std=0.0076

Evaluation Metrics Random Forest chi2 k = 5000
Random Forest Accuracy: 0.4349
Classification report:
              precision    recall  f1-score   support

           1       0.71      0.07      0.13       142
           2       0.36      0.09      0.14       235
           3       0.44      0.93      0.59       392
           4       0.39      0.06      0.10       160

    accuracy                           0.43       929
   macro avg       0.48      0.29      0.24       929
weighted avg       0.45      0.43      0.32       929

Macro F1-score: 0.23961891575367214
Weighted F1-score: 0.3220627915021907
```



```
CV Linear SVC (k=5000) F1 macro per fold: [0.38926619 0.41247473 0.38431984
0.39520251 0.4079629 ]
CV Linear SVC (k=5000) mean=0.3978, std=0.0108

Evaluation Metrics Linear SVC chi2 k = 5000
Linear SVC Accuracy: 0.4284
Classification report:
              precision    recall  f1-score   support

           1       0.39      0.33      0.36       142
           2       0.35      0.33      0.34       235
           3       0.52      0.57      0.54       392
           4       0.32      0.31      0.31       160

    accuracy                           0.43       929
   macro avg       0.39      0.39      0.39       929
weighted avg       0.42      0.43      0.42       929

Macro F1-score: 0.38879530272059093
Weighted F1-score: 0.42416873232775454
```



[ ]:
```python
# add user feature to the best performing model
import numpy as np
from scipy.sparse import hstack
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

def _build_dense_feature(series, feature_name: str = "useful"):
    """Create a dense column vector from a single feature, filling NaNs with 0.
    """
    return (
        series[[feature_name]]
        .fillna(0)
        .astype(float)
        .values
        .reshape(-1, 1)
    )

# Precompute dense "useful" feature for train/test
X_train_useful = _build_dense_feature(X_train, "useful")
X_test_useful = _build_dense_feature(X_test, "useful")

for k in k_values:
    # Combine user feature with chi2-selected TF-IDF feature vector
    X_train_combined = hstack([X_train_chi2_selection[k], X_train_useful])
    X_test_combined = hstack([X_test_chi2_selection[k], X_test_useful])

    # Logistic Regression
    logistic_regression_model = LogisticRegression(
        max_iter=2000,
        n_jobs=-1,
        solver="saga",
        penalty="l2",
        class_weight="balanced",
    )
    y_pred = evaluate_model(
        logistic_regression_model,
        "Logistic Regression",
        X_train_combined,
        y_train_combined,
        X_test_combined,
        y_test,
        k,
    )
    plot_confusion_matrices("Logistic Regression", y_test, y_pred, k)

    # LinearSVC
    linear_svc_model = LinearSVC(class_weight="balanced")
```

```
    y_pred = evaluate_model(
        linear_svc_model,
        "LinearSVC",
        X_train_combined,
        y_train_combined,
        X_test_combined,
        y_test,
        k,
    )
    plot_confusion_matrices("LinearSVC", y_test, y_pred, k)
```